RELATÓRIO TÉCNICO

Francisco Costa Carneiro

linkedin.com/in/franciscocostacarneiro

RFB CRAWLER

Extração automatizada de dados públicos da Receita Federal do Brasil

Documento Técnico — Versão 2.0

Brasília, DF
2026

1. VISÃO GERAL

O **RFB Crawler** é uma aplicação Python que extrai automaticamente os dados públicos disponibilizados pela Receita Federal do Brasil (RFB), organizando-os localmente com controle de deduplicação, retenção e auditoria. O resultado final é um executável Windows autossuficiente (`RFB_Crawler.exe`) que não exige instalação do Python no computador do usuário.

---

2. FONTE DE DADOS

Portal Ativo: Nextcloud da Receita Federal

- **URL:** `https://arquivos.receitafederal.gov.br`
- **Tipo:** Nextcloud (servidor de arquivos colaborativo baseado em PHP/SabreDAV)
- **Compartilhamento público:** token `gn672Ad4CF8N6TK`
- **Caminho raiz dos dados cadastrais:** `/Dados/Cadastros`

Datasets disponíveis

Dataset	Descrição	Caminho no servidor
CNPJ	Cadastro Nacional de Pessoas Jurídicas — empresas, sócios, estabelecimentos, CNAEs, etc.	`/Dados/Cadastros/CNPJ`
CAFIR	Cadastro de Imóveis Rurais	`/Dados/Cadastros/CAFIR`
CNO	Cadastro Nacional de Obras	`/Dados/Cadastros/CNO`
SISEN	Sistema de Entidades Sem Fins Lucrativos	`/Dados/Cadastros/SISEN`

Cada dataset é organizado em subpastas por **competência** (ex.: `2026-04`), que correspondem ao mês de referência da extração. Dentro de cada competência há arquivos `.zip` com os dados tabulares.

Portal Legado (inativo)

O portal Apache `https://dadosabertos.rfb.gov.br` era a fonte original, mas encontra-se inacessível (TCP timeout nas portas 80 e 443). O código mantém suporte para reativação futura via `PORTAL = "apache"` em `config.py`.

Protocolo de acesso: WebDAV

O Nextcloud expõe um endpoint WebDAV público em `/public.php/webdav/`. A navegação de pastas é feita via requisições **PROPFIND** (método HTTP não-padrão

do protocolo WebDAV/RFC 4918) com autenticação Basic `(token, "")`. O download dos arquivos é feito via **GET** na mesma rota WebDAV.

---

3. ARQUITETURA DA APLICAÇÃO

3.1 Visão de Camadas

```
┌──────────────────────────────────────┐
│          GUI / Ponto de Entrada      │
│   rfb_crawler.py  (EXE standalone)   │
│   crawler/main.py (módulo Python)    │
└──────────────────────────────────────┘
                │
                │
┌───────────────▼──────────────────────┐
│           Orquestrador               │
│   ThreadPoolExecutor (3 workers)     │
│   Coleta entradas → dispara          │
│   downloads em paralelo              │
└──────────────────────────────────────┘
                │
        ┌───────┴────────┐
        │                │
┌───────▼────────┐  ┌────▼──────────────┐
│   Scraper      │  │   NetworkSession  │
│ PROPFIND DFS   │  │ GET streaming     │
│ DirectoryScraper│ │ PROPFIND helper   │
│ NextcloudScraper│ │ Retry + backoff   │
└────────────────┘  └───────────────────┘
        │                │
┌───────▼────────────────▼──────────────┐
│          DataGovernance               │
│  manifest.json  catalog.json          │
│  execution_log.csv  retention         │
└──────────────────────────────────────┘
```

3.2 Módulos

Arquivo	Responsabilidade
`rfb_crawler.py`	Versão monolítica compilada para EXE. Contém toda a lógica + GUI Tkinter.
`crawler/config.py`	Constantes tunáveis: URLs, token, datasets, workers, timeout, retenção.
`crawler/network.py`	`NetworkSession`: sessão HTTP com retries, `stream_download()`, `propfind()`.
`crawler/scraper.py`	`DirectoryScraper` (Apache HTML) e `NextcloudScraper` (WebDAV). DFS recursivo.
`crawler/filesystem.py`	`DataGovernance`: manifesto SHA-256, catálogo, log CSV, política de retenção.
`crawler/main.py`	Orquestrador: `ThreadPoolExecutor`, resumo de saúde, webhook.
`crawler/exceptions.py`	`RFBConnectionError`, `DiskFullError`, `SchemaChangeError`.
`build_exe.bat`	Script de compilação via PyInstaller.
`rfb_crawler.spec`	Especificação do PyInstaller (imports ocultos, ícone, nome do EXE).

---

4. FLUXO DE EXECUÇÃO

```
Usuário executa RFB_Crawler.exe
        │
        ▼
[GUI] Escolhe pasta de destino (tkinter.filedialog)
        │
        ▼
[GUI] Seleciona portal: Apache ou Nextcloud
        │
        ▼
[Verificação] HEAD request → conectividade OK?
   NÃO → oferece configuração de proxy → tenta novamente → aborta com mensag
        │
        ▼ SIM
[GUI] Lista datasets disponíveis (PROPFIND ou HTML scrape)
        │
        ▼
[GUI] Usuário seleciona datasets e subpastas (competências)
        │
        ▼
[GUI] Confirmação do resumo
        │
        ▼
[Crawler] Para cada dataset selecionado:
   1. DFS via PROPFIND → lista todos os arquivos recursivamente
   2. ThreadPoolExecutor (3 workers) → baixa arquivos em paralelo
      ├─ Nível 1: metadata WebDAV (tamanho + modificado) bate com manifesto?
      ├─ Nível 2: arquivo existe localmente? → calcula SHA-256 → está no man
      ├─ Download streaming em chunks de 8 MB → arquivo .part → renomeia ao
      ├─ Nível 3: hash pós-download já conhecido (conteúdo idêntico, outro n
      └─ Registra no manifesto (com remote_size_bytes + remote_modified) e n
   3. Aplica política de retenção (mantém últimas 3 competências)
        │
        ▼
[Persistência] Salva manifest.json, catalog.json, execution_log.csv
        │
        ▼
[Saída] Resumo no console: GB baixados, arquivos novos, skips, erros
        │
        ▼
[GUI] Nenhum arquivo novo E sem erros?
   SIM → caixa de diálogo: "Base já atualizada — nada para incrementar"
   NÃO → encerra normalmente
```

---

5. TECNOLOGIAS E SKILLS UTILIZADAS

5.1 Python 3.14

Linguagem principal. Recursos utilizados: - **Type hints** com `|` union syntax (`str | None`), `tuple[str, str]` - **Dataclasses** (`@dataclass`, `field(default_factory=...)`) para `DirectoryEntry` - **`__future__.annotations`** para avaliação lazy de anotações - **`pathlib.Path`** para manipulação de caminhos

multiplataforma - `concurrent.futures.ThreadPoolExecutor` para paralelismo de I/O - `hashlib.sha256` para verificação de integridade - `xml.etree.ElementTree` para parsing de respostas WebDAV (XML) - `tkinter` para GUI nativa (seleção de pasta, caixas de diálogo, listbox multi-seleção) - `csv.DictWriter` para log de execuções - `shutil.disk_usage` para detecção de disco cheio - **Generator functions** (`yield from`) para DFS com baixo consumo de memória

5.2 requests + urllib3

Biblioteca HTTP de alto nível. Recursos utilizados: - `HTTPAdapter` + `Retry`: backoff exponencial, 3 tentativas em erros 5xx - `stream=True` + `iter_content(chunk_size)`: download de arquivos grandes sem carregar na memória (streaming) - `session.request("PROPFIND", ...)`: método HTTP personalizado para WebDAV - `auth=(token, "")`: autenticação Basic para acesso ao Nextcloud

5.3 BeautifulSoup4 + lxml

Parsing do HTML de listagens Apache. O parser `lxml` é usado como backend para maior velocidade e tolerância a HTML mal-formado. Estratégia de fallback: `table tr` → `pre` → todos os `<a>`.

5.4 WebDAV (RFC 4918)

Protocolo de extensão do HTTP para sistemas de arquivos remotos. Conceitos aplicados: - **PROPFIND com** `Depth: 1`: lista o conteúdo de um diretório sem recursão (controlada manualmente via DFS) - **Namespace XML** `DAV:`: parsing de `d:response`, `d:href`, `d:resourcetype`, `d:collection`, `d:getcontentlength` - `/public.php/webdav/`: endpoint do SabreDAV (engine WebDAV do Nextcloud) para compartilhamentos públicos

5.5 PyInstaller

Empacotamento do script Python em executável Windows autossuficiente. Configurações no `rfb_crawler.spec`: - Arquivo de entrada: `rfb_crawler.py` - Hidden imports declarados explicitamente: `requests`, `urllib3`, `certifi`, `bs4`, `lxml`, `tqdm`, `tkinter` - Modo one-file EXE com ícone personalizado

5.6 Padrões de design aplicados

- **Single Responsibility Principle**: cada módulo tem uma única responsabilidade bem definida
- **Generator + DFS**: o crawler não acumula todos os caminhos em memória — produz entradas sob demanda via `yield from`

- **Idempotência**: verificação por SHA-256 antes de qualquer download evita retrabalho
- **Atomic write**: o arquivo é escrito em `.part` e só renomeado ao concluir, evitando arquivos corrompidos por interrupção
- **Retry com backoff exponencial**: `backoff_factor=2` → esperas de 2s, 4s, 8s entre tentativas

---

6. ESTRUTURA DE SAÍDA (PASTA `DATA/`)

```
data/
├── manifest.json          # Hash SHA-256 → metadados de todos os arquivos b
├── catalog.json           # Índice público regenerado a cada execução
├── execution_log.csv      # Histórico de todas as execuções (run_id, status
├── CNPJ/
│   ├── 2026-02/
│   │   ├── Empresas0.zip
│   │   └── ...
│   ├── 2026-03/
│   └── 2026-04/           ← competência mais recente
│       ├── Cnaes.zip
│       ├── Empresas0.zip
│       └── ...
├── CAFIR/
├── CNO/
└── SISEN/
```

manifest.json (exemplo de entrada)

```
{
  "5c20d0c74a39b764af47ff76de36b063...": {
    "name": "Cnaes.zip",
    "source_url": "https://arquivos.receitafederal.gov.br/public.php/webdav/
    "parent_folder": "2026-04",
    "hierarchy": ["CNPJ", "2026-04"],
    "local_path": "CNPJ/2026-04/Cnaes.zip",
    "size_bytes": 22078,
    "remote_size_bytes": 22078,
    "remote_modified": "Wed, 10 Apr 2026 02:15:33 GMT",
    "file_hash": "5c20d0c74a39b764af47ff76de36b063...",
    "download_timestamp": "2026-04-23T15:30:00+00:00"
  }
}
```

Os campos `remote_size_bytes` e `remote_modified` são persistidos para viabilizar o **Nível 1** da extração incremental (ver seção 12).

execution_log.csv (campos)

`run_id`, `dataset`, `file_name`, `status` (new/skipped/error), `source_url`, `parent_folder`, `size_bytes`, `file_hash`, `download_timestamp`, `error_message`

---

7. POLÍTICA DE RETENÇÃO

Por padrão (`RETENTION_COUNT = 3`), após cada execução o sistema mantém apenas as **3 competências mais recentes** por dataset, removendo as mais antigas via `shutil.rmtree`. O manifesto é atualizado em memória para remover as entradas dos arquivos deletados.

---

8. CONFIGURAÇÃO (`CRAWLER/CONFIG.PY`)

Constante	Valor padrão	Descrição
`PORTAL`	`"nextcloud"`	Portal ativo (`"apache"` ou `"nextcloud"`)
`NC_BASE_URL`	`https://arquivos.receitafederal.gov.br`	URL base do Nextcloud
`NC_SHARE_TOKEN`	`gn672Ad4CF8N6TK`	Token do compartilhamento público
`NC_SHARE_ROOT`	`/Dados/Cadastros`	Caminho raiz dentro do share
`DATASETS`	`["CNPJ","CAFIR","CNO","SISEN"]`	Datasets a crawlear
`MAX_WORKERS`	`3`	Workers paralelos de download
`CHUNK_SIZE`	`8 MB`	Tamanho do chunk de streaming
`TIMEOUT`	`(10, 60)`	Timeout de conexão e leitura (segundos)
`MAX_RETRIES`	`3`	Tentativas em erros transitórios
`RETENTION_COUNT`	`3`	Competências a manter por dataset

---

9. DEPENDÊNCIAS

Pacote	Versão mínima	Uso
`requests`	2.32.0	HTTP, streaming, PROPFIND
`beautifulsoup4`	4.12.0	Parsing de HTML (portal Apache)
`lxml`	5.2.0	Parser HTML de alto desempenho
`tqdm`	4.66.0	Barras de progresso (reservado)
`pyinstaller`	6.x	Compilação do EXE (dev only)

Todas as demais dependências (`urllib3`, `certifi`, `charset-normalizer`, `idna`) são transitivas do `requests`.

---

10. COMO EXECUTAR

Via EXE (usuário final)

```
dist\RFB_Crawler.exe
```

1. Selecionar pasta de destino
2. Escolher portal (Nextcloud recomendado)
3. Selecionar datasets e competências
4. Confirmar e aguardar

Via Python (desenvolvedor)

```
# Ativar venv
venv\Scripts\activate

# Todos os datasets (Nextcloud)
python -m crawler.main

# Dataset específico
python -c "from crawler.main import run; run(datasets=['CNPJ'])"

# Portal Apache (quando disponível)
python -c "from crawler.main import run; run(portal='apache')"
```

Recompilar o EXE

```
build_exe.bat
```

---

11. TRATAMENTO DE ERROS

Exceção	Causa	Comportamento
`RFBConnectionError`	Timeout, conexão recusada, HTTP 4xx/5xx	Log de erro; arquivo marcado como `error` no CSV; execução continua
`DiskFullError`	Espaço livre < tamanho do chunk	Log crítico; arquivo `.part` removido; execução continua
`SchemaChangeError`	Diretório raiz do dataset retornou vazio	Log de erro; dataset inteiro ignorado

Em caso de interrupção durante o download, o arquivo `.part` é removido automaticamente, garantindo que nunca haja dados corrompidos em disco.

---

12. EXTRAÇÃO INCREMENTAL

O RFB Crawler implementa uma estratégia de **três níveis de verificação** para evitar downloads redundantes. Na segunda execução em diante, arquivos já baixados são identificados e pulados sem consumir banda ou processamento desnecessários.

12.1 Visão geral do mecanismo

Ao inicializar, `DataGovernance` lê o `manifest.json` e constrói dois índices em memória:

Índice	Chave	Valor	Uso
`_manifest`	SHA-256 do arquivo	registro completo	Nível 2 e 3
`_url_index`	URL de origem (`source_url`)	registro completo	Nível 1

O `_url_index` é um índice secundário criado pelo método `_build_url_index()`. Ele permite buscar em O(1) se uma URL já foi baixada anteriormente, sem precisar iterar todo o manifesto.

12.2 Nível 1 — Comparação de metadados WebDAV (caminho rápido)

Método: `DataGovernance.is_unchanged(entry: DirectoryEntry) → bool`

Compara os metadados retornados pelo servidor WebDAV (via PROPFIND) com os valores armazenados no manifesto:

1. Busca a `entry.url` no `_url_index`
2. Compara `entry.size_bytes` com `rec["remote_size_bytes"]`
3. Compara `entry.modified` com `rec["remote_modified"]`
4. Verifica se o arquivo local ainda existe em disco

Se **todos** os critérios baterem, o arquivo é considerado **inalterado** e pulado **sem nenhuma leitura de disco** (zero I/O além da verificação de existência do caminho). Esta é a rota mais eficiente.

```
entry.url → _url_index → rec?
                         |
                    NÃO ⟶| (prossegue para download)
                         |
                    SIM ⟶| rec["remote_size_bytes"] == entry.size_bytes?
                         | rec["remote_modified"]  == entry.modified?
                         | local_path.exists()?
                         |
                    TODOS SIM ⟶ SKIP (status: skipped)
                    QUALQUER NÃO ⟶ prossegue
```

12.3 Nível 2 — Verificação por SHA-256 local (fallback)

Método: `DataGovernance.is_known(file_hash: str) → bool`

Se o Nível 1 falhar (ex.: arquivo baixado antes da adição dos campos `remote_size_bytes`/`remote_modified` ao manifesto, ou metadata divergente por mudança de fuso horário do servidor), o crawler tenta um caminho alternativo:

1. Verifica se o arquivo existe localmente (`local_path.exists()`)
2. Calcula o SHA-256 do arquivo local (`_sha256_file()`)
3. Busca o hash no `_manifest`

Se o hash estiver no manifesto, o arquivo é considerado já baixado e é pulado. Esta verificação é mais lenta que o Nível 1 por exigir leitura completa do arquivo em disco.

12.4 Nível 3 — Deduplicação pós-download por hash

Após um download completo, o hash SHA-256 do arquivo recém-baixado é calculado e verificado no `_manifest`:

- Se o hash **já existir**: o arquivo baixado é descartado (`local_path.unlink()`) e registrado como `skipped`. Isso cobre o cenário em que o mesmo conteúdo está disponível sob diferentes nomes ou caminhos no servidor.
- Se o hash **não existir**: o arquivo é registrado no manifesto via `register_file()`, que salva também `remote_size_bytes` e `remote_modified` para habilitar o Nível 1 nas próximas execuções.

12.5 Diagrama de decisão por arquivo

```
┌──────────────────────────────────────────────────────────┐
│   Para cada arquivo listado pelo PROPFIND                │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
            ┌──────────────────────────┐
            │  Nível 1                 │
            │  is_unchanged(entry)     │
            │  (URL + size + date      │
            │   + file exists?)        │
            └──────────────────────────┘
                 SIM ──┤      NÃO
                       │        │
              ┌────────┴──┐  ┌──▼──────────────────┐
              │  SKIP     │  │  Nível 2            │
              │(skipped)  │  │  local_path.exists()?  │
              └───────────┘  │  SHA-256 in manifest? │
                             └─────────────────────┘
                        SIM ──┤      NÃO
                              │        │
                     ┌────────┴──┐  ┌──▼──────────────────┐
                     │  SKIP     │  │  Download streaming  │
                     │(skipped)  │  │  (chunks de 8 MB)    │
                     └───────────┘  └─────────────────────┘
                                          │
                                  ┌──────▼──────────────┐
                                  │  Nível 3            │
                                  │  SHA-256 in manifest?  │
                                  └─────────────────────┘
                                     SIM ──┤      NÃO
                                           │        │
                                  ┌────────┴──┐  ┌──▼──────────────────┐
                                  │  SKIP     │  │  register_file()    │
                                  │(skipped)  │  │  status: new        │
                                  └───────────┘  └─────────────────────┘
```

12.6 Persistência dos metadados remotos

O método `register_file()` em `DataGovernance` persiste dois campos adicionais no manifesto para cada arquivo novo:

```
"remote_size_bytes": entry.size_bytes,   # Content-Length do PROPFIND
"remote_modified":  entry.modified,      # Last-Modified do PROPFIND
```

Esses campos são a base do Nível 1. Arquivos baixados por versões anteriores do crawler (sem esses campos) continuarão sendo verificados pelo Nível 2 até que sejam re-baixados e o manifesto seja atualizado.

12.7 Notificação de base completa

Ao final de cada execução, a função `run()` avalia os totais acumulados:

```
if total_new == 0 and total_err == 0:
    messagebox.showinfo(
        "RFB Crawler — Base já atualizada",
        "Nenhum arquivo novo foi encontrado ...\n"
        f"Arquivos verificados: {total_skip}\n"
        "Não há nada para incrementar.",
    )
```

A condição exige **simultaneamente**: - `total_new == 0` — nenhum arquivo foi baixado/registrado nesta execução - `total_err == 0` — nenhum erro ocorreu (garante que o zero de novos não seja resultado de falhas de rede)

Se qualquer erro tiver ocorrido, o diálogo **não** é exibido, pois arquivos podem ter deixado de ser baixados por falha, e não por já estarem atualizados.

O bloco é protegido por `try/except Exception`, portanto em contextos sem interface gráfica (ex.: execução via linha de comando com `python -m crawler.main`) a exceção é silenciada e o programa encerra normalmente.

12.8 Comportamento na segunda execução

Em um cenário típico onde o servidor não publicou novos dados:

Fase	Tempo estimado	I/O de rede	I/O de disco
PROPFIND (DFS)	~2–5 s	Mínimo (XML leve)	Nenhum
Verificação Nível 1 (por arquivo)	< 1 ms por arquivo	Nenhum	Nenhum
Total para datasets sem mudanças	**~5–10 s**	**Mínimo**	**Nenhum**

Isso contrasta com uma primeira execução, que pode levar horas dependendo do volume de dados (os datasets CNPJ possuem dezenas de GBs).